Mid Level Developer Practical Test

To-Do Board Solution

Your task is to develop a To-Do Board with the following key features:

Core Features:

1. **Authentication**
   a. Implement authentication in your application following best practices.
   b. Having the ability to register users is a bonus.
2. **Task Management Features**
   a. Users should be able to create categories/labels (e.g., "To Do," "In Progress," "Done").
   b. Users should be able to create tickets (tasks) and assign them to a category/label.
   c. Each ticket should have:
      i. Title (required)
      ii. Description (editable & auto-saved as a draft when interrupted)
      iii. Expiry Date (with an intuitive notification when near expiration)
3. **Drag-and-Drop Board (Without External Libraries)**
   a. Implement a drag-and-drop functionality without using a library (e.g., no react-beautiful-dnd).
   b. Use the HTML Drag and Drop API and handle events such as dragstart, dragover, drop, etc.
   c. Ensure a smooth user experience similar to Trello/Jira.
4. **Ticket Editing & Draft Saving**
   a. When a ticket's expiry date is approaching, the system should notify the user in a user-friendly way.
   b. Suggestions for notifications:
      i. Visual indicators (badges, color changes)
      ii. Toast messages
      iii. **Bonus**: Email or push notifications
   c. **Bonus:** Track Update History – Maintain a record of changes, showing when a ticket was moved.

Technical Requirements

Front-End (FE)

- Use Next.js, React, Vue.js, or Angular (**Highly recommend React**).
- Style the application using SASS, LESS, or Tailwind CSS.
- Good UX/UI sense must be clearly demonstrated on the end product
- Ensure state management is handled properly (e.g., React Context, Redux).

Data Fetching

- For data fetching create a simple API that can serve data to your forntend
  - You can use any back-end framework you're comfortable with (Node.js, Express, NestJS, .NET Core, etc.) to communicate with DB and Fetch Data

- Use any Relational or Document Database (We recommend MySQL)

Submission Guidelines

- Submit your project via a publicly accessible Git repository.
- The repository must show the development process (no single commit repos).
- Follow best Git practices:
  - Meaningful commit messages
  - Feature branches
- Include a README.mdwith:
  - Installation instructions
  - Setup guide
  - Usage instructions
- **Bonus:** Dockerize your application for easy setup.

Timeframe:

You have **15 days** from receiving this test to submit your work.

Good Luck, and if you have any questions, feel free to email info@techlint.com anytime!